Operações, contêineres e Kubernetes

Como manter um serviço funcionando enquanto tudo muda

Prof. Lucca Abbado Neres

Centro Universitário Fundação Assis Gurgacz – FAG

DevOps | 2026



O sistema abriu. Mas ele aguenta o dia inteiro?

Uma transportadora usa um sistema para planejar entregas. Pela manhã, dez pessoas consultam rotas. Depois de uma promoção, centenas precisam do serviço ao mesmo tempo.

A pergunta de quem usa

"Consigo calcular minha rota agora, sem esperar demais?"

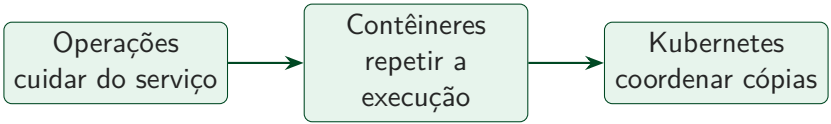
A pergunta de quem opera

"Temos capacidade? Como detectar falhas? Como mudar o sistema sem interromper o trabalho?"

Ideia principal

Uma funcionalidade só entrega valor quando pode ser usada. É aí que começa a operação de sistemas.

O caminho da aula

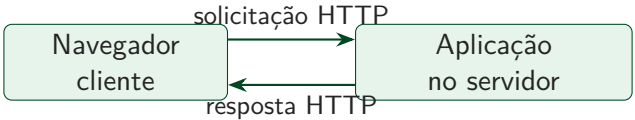


Vamos começar pela aplicação e pelo servidor. Depois, veremos como observar o serviço, lidar com falhas e publicar mudanças. Os nomes das ferramentas aparecerão quando houver um problema para resolver.

Ideia principal

Ao final, será possível explicar por que colocar mais cópias de uma aplicação pode ajudar – e por que isso nem sempre basta.

O que acontece quando clicamos em “calcular”?



O cliente pede um trabalho. A CPU faz o processamento; a memória mantém informações em uso; a rede transporta os pedidos e as respostas. A aplicação usa esses recursos do servidor para devolver uma página, dados ou um erro.

Em palavras simples

Servidor é o computador que oferece recursos. Aplicação é o programa. API é uma interface pela qual programas fazem pedidos uns aos outros. HTTP é um protocolo usado nessa conversa.

Desenvolvimento e produção têm preocupações diferentes

Produção é o ambiente usado por pessoas e processos reais. Uma falha pode impedir uma entrega, uma venda ou um atendimento.

Durante o desenvolvimento	Durante a operação
A rota é calculada corretamente?	A rota continua disponível nos horários de pico?
A nova versão funciona no teste?	Como publicá-la e voltar atrás se houver problema?
O programa inicia na minha máquina?	Outra pessoa consegue executar e recuperar o serviço?

Ideia principal

Desenvolver e operar são responsabilidades conectadas durante toda a vida do sistema.

Ops: cuidar do serviço em funcionamento

Em palavras simples

Operações é o trabalho de manter o sistema útil, acessível e recuperável para quem depende dele.

Responsabilidade	**Exemplo na transportadora**
Disponibilidade	Perceber que o serviço caiu e restaurar o atendimento.
Desempenho e capacidade	Investigar lentidão e preparar recursos para o pico.
Mudanças	Publicar versões com verificação e possibilidade de retorno.
Segurança	Controlar acessos e corrigir componentes vulneráveis.
Continuidade	Fazer cópias de segurança e testar a restauração.
Custos	Evitar pagar por capacidade ociosa sem necessidade.

O trabalho de Ops não começa quando algo quebra

Boa parte do trabalho acontece antes do incidente: acompanhar tendências, automatizar tarefas repetidas, testar recuperação e revisar mudanças.

Trabalho reativo

Investigar um alerta, restaurar uma dependência, comunicar o impacto e recuperar o serviço.

Trabalho preventivo

Melhorar alarmes, documentar procedimentos, corrigir causas recorrentes e planejar capacidade.

Ideia principal

Reiniciar manualmente o mesmo serviço todo dia pode resolver o sintoma. Descobrir por que ele precisa ser reiniciado reduz o trabalho futuro.

Fonte: Google SRE, Eliminating Toil. Consulta: 15 out. 2026.

Quatro sinais para saber como o serviço está

Latência: quanto demora?

Tempo para responder a uma solicitação. Uma rota que leva vários segundos pode atrapalhar o atendimento.

Tráfego: quanto chega?

Volume de solicitações por segundo. É a demanda que o sistema precisa atender.

Erros: o que não deu certo?

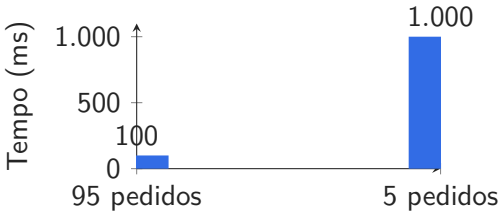
Parcela de solicitações que falham ou entregam um resultado inválido. Responder rápido com erro não é atender bem.

Saturação: o que está no limite?

Recursos próximos da capacidade: CPU ocupada, memória esgotada ou filas crescendo.

Fonte: Google SRE, quatro sinais de monitoramento. Consulta: 15 out. 2026.

A média pode esconder quem está esperando



Neste exemplo, a média é 145 ms. Mas cinco pessoas esperaram um segundo.

Em palavras simples

O percentil 95, ou p95, descreve um tempo abaixo do qual ficam aproximadamente 95% das observações. Ele ajuda a enxergar a parte lenta do atendimento.

Ideia principal

Leia o p95 junto da taxa de erros e do número de respostas. Um teste com quase nenhum sucesso não pode ser considerado bom.

Dados ilustrativos, não provenientes de pesquisa ou medição. Conceito: Google SRE, Monitoring Distributed Systems.

Como definir o que significa “funcionar bem”?

Uma meta torna a conversa objetiva. Em vez de “o sistema precisa ser rápido”, podemos combinar qual operação será medida, em qual período e com qual limite.

SLI: o indicador observado

Exemplo: proporção de cálculos de rota concluídos com sucesso durante o mês.

SLO: o objetivo combinado

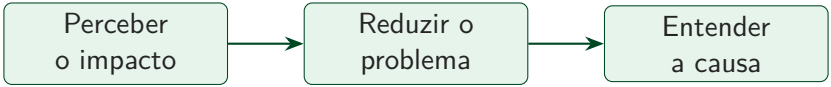
Exemplo: pelo menos 99,5% desses pedidos devem ter sucesso no mês. É uma meta de negócio, não uma regra universal.

Ideia principal

Um objetivo mais exigente pode exigir mais recursos e trabalho operacional. A escolha depende do impacto das falhas. SLA é um acordo de serviço, que pode incluir consequências contratuais.

Fonte: Google SRE, Service Level Objectives. Consulta: 15 out. 2026.

Quando há um incidente, primeiro recupere o atendimento



Se uma versão nova começou a produzir erros, retornar à versão anterior pode recuperar o serviço antes de a investigação terminar. Depois, logs, métricas e mudanças recentes ajudam a explicar o ocorrido.

Durante o incidente

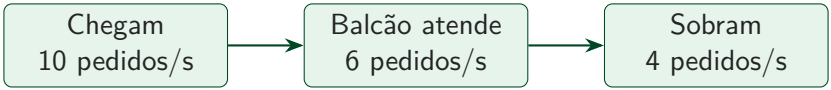
Verificar quem foi afetado, comunicar a situação e escolher uma ação de recuperação com risco controlado.

Depois do incidente

Registrar o que aconteceu e melhorar o sistema: teste ausente, alerta tardio, limite inadequado ou procedimento confuso.

Fluxo conceitual de resposta a incidentes. As ações dependem do sistema e das responsabilidades da equipe.

Capacidade: não cabem infinitos pedidos no mesmo balcão



Se o excesso entra em uma fila, a espera cresce. Se a aplicação não aceita mais trabalho, parte dos pedidos falha. Nos dois casos, a demanda ultrapassou a capacidade naquele cenário.

Em palavras simples

Capacidade é quanto trabalho o sistema consegue atender mantendo a qualidade esperada. Um gargalo é a parte que limita o conjunto.

Ideia principal

Aumentar CPU pode ajudar em um cálculo pesado. Se a espera está no banco de dados, mais cópias da aplicação podem apenas aumentar a pressão sobre o banco.

Exemplo numérico ilustrativo: demanda constante, sem mudança de capacidade.

Operar também é proteger e conseguir recuperar

Acesso e atualização

Cada pessoa e serviço precisa apenas das permissões necessárias. Imagens, bibliotecas e sistemas precisam de atualização e acompanhamento.

Dados e recuperação

Backup guarda uma cópia. Testar a restauração verifica se ela realmente pode devolver o serviço a um estado utilizável.

Na analogia da transportadora, abrir outro balcão não recupera o cadastro perdido. Cópias da aplicação e cópias dos dados resolvem problemas diferentes.

Ideia principal

Disponibilidade, segurança e recuperação precisam ser consideradas juntas. Reiniciar um contêiner não substitui uma estratégia de proteção dos dados.

DevOps: aproximar quem constrói de quem opera

Em palavras simples

DevOps é trabalhar com responsabilidade compartilhada para entregar mudanças e manter o serviço confiável. O retorno da operação ajuda a melhorar o desenvolvimento.

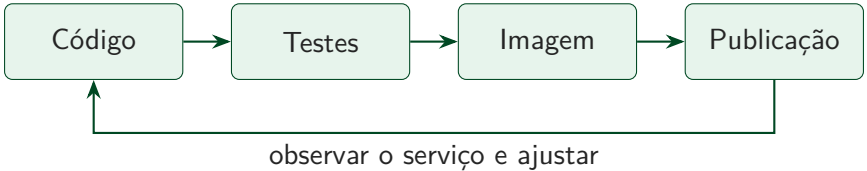
Definição técnica

É um conjunto de práticas culturais e técnicas que integra desenvolvimento e operações por colaboração, automação e ciclos rápidos de feedback. Não se resume a um cargo ou uma ferramenta.

Uma melhoria concreta: quem desenvolve fornece uma verificação de saúde; quem opera usa essa informação para identificar instâncias que podem receber tráfego.

Fonte: DORA, capacidades de entrega de software. Consulta: 15 out. 2026.

Automação transforma uma mudança em um caminho repetível



Uma pipeline é uma sequência automatizada de etapas. Ela pode testar a aplicação, produzir um pacote e publicá-lo de forma consistente.

Em palavras simples

Integração contínua verifica mudanças com frequência. Entrega contínua mantém o software em condição de ser publicado. A publicação automática em produção é uma escolha adicional.

A pipeline depende de critérios de aceitação, permissões e procedimentos de recuperação.

O que a pesquisa DORA ajuda a observar?

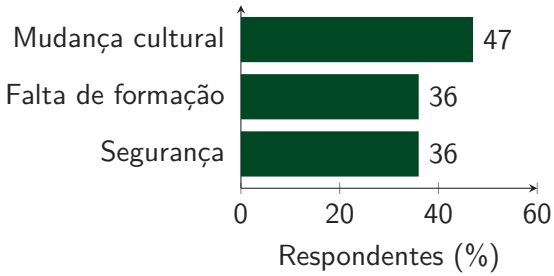
A DORA pesquisa a entrega de software. Suas métricas permitem discutir velocidade e instabilidade de mudanças no mesmo serviço.

Métrica	**Pergunta que ela ajuda a responder**
Frequência de implantação	Com que frequência publicamos?
Tempo de entrega da mudança	Quanto leva do commit até a produção?
Taxa de falha de mudanças	Quantas implantações exigem intervenção?
Recuperação de implantação falha	Quanto leva para recuperar uma mudança malsucedida?
Retrabalho de implantação	Quanto publicamos para corrigir problemas de mudanças?

Essas medidas descrevem o sistema de trabalho; não servem para classificar pessoas pela quantidade de comandos executados.

Fonte: DORA, cinco métricas de desempenho de entrega. Consulta: 15 out. 2026.

As dificuldades também envolvem pessoas e conhecimento



No levantamento CNCF de 2025, estes foram desafios citados no uso de contêineres.

Ideia principal

Aprender os conceitos e combinar responsabilidades faz parte da adoção. Instalar uma ferramenta não resolve tudo.

Fonte: CNCF Annual Survey 2025, fig. 7, p. 10; publicado em jan. 2026. Base: 351 respostas de organizações usuárias; múltipla seleção, sem "não sei".

Recorte da comunidade cloud native, não de todas as empresas.

Antes dos contêineres: “aqui funciona”

Uma aplicação depende de mais que seu código: versão da linguagem, bibliotecas, arquivos e configuração. Se cada ambiente é montado manualmente, essas peças podem ficar diferentes.

Na máquina de desenvolvimento

Python e bibliotecas de uma versão. Certos arquivos estão presentes e o caminho esperado existe.

No servidor

Outra versão de biblioteca, um arquivo ausente ou uma configuração diferente. O mesmo código pode falhar.

Em palavras simples

É como enviar uma receita sem combinar ingredientes e equipamentos. O resultado depende do que cada cozinha tem disponível.

Contêiner: um processo em um ambiente isolado

Em palavras simples

Um contêiner reúne as condições para executar uma aplicação de maneira mais previsível, separando seu ambiente do de outros programas.

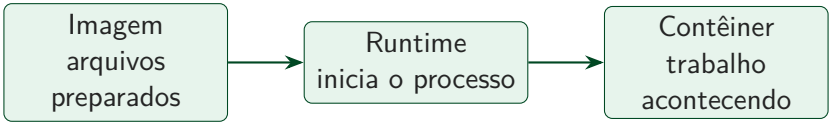
Definição técnica

É uma execução isolada de um ou mais processos, com sistema de arquivos e dependências próprios. No caso de contêineres Linux, mecanismos do kernel isolam recursos e controlam seu uso.

Pense em uma estação de trabalho organizada dentro de uma cozinha compartilhada. Cada estação tem seus utensílios, mas depende da infraestrutura comum. O isolamento não é uma barreira de segurança absoluta.

Fonte: Docker Docs, What is a container?. Consulta: 15 out. 2026.

Imagem é o pacote; contêiner é a execução



Uma imagem é um modelo de arquivos e metadados usado para iniciar contêineres. O runtime é o componente que os executa. A mesma imagem pode iniciar várias execuções independentes.

Em palavras simples

A imagem se parece com um kit padronizado para montar a estação. O contêiner é a estação montada e trabalhando. Um registry armazena e distribui esses kits.

Identificar a versão da imagem ajuda a publicar o mesmo pacote em teste e produção.

Fonte: Docker Docs, What is an image?. Consulta: 15 out. 2026.

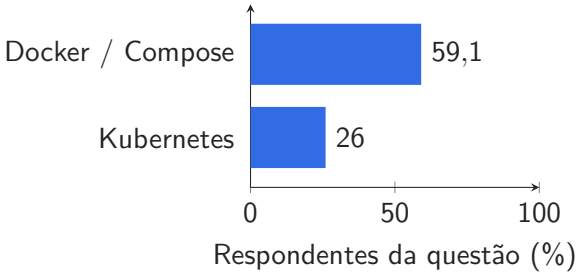
Contêiner e máquina virtual não são a mesma coisa

	Máquina virtual	**Contêiner Linux**
O que é isolado	Um sistema operacional convidado.	Processos e seu ambiente.
Kernel	Cada VM tem seu próprio kernel.	Compartilha o kernel do hospedeiro.
Uso comum	Separar sistemas operacionais e máquinas.	Empacotar e executar aplicações.
Relação	Pode hospedar vários contêineres.	Pode executar dentro de uma VM.

No Windows ou macOS, ferramentas como Docker Desktop podem usar uma máquina virtual Linux para executar contêineres Linux. As duas tecnologias se complementam.

Fonte: Docker Docs, contêineres e máquinas virtuais. Consulta: 15 out. 2026.

Por que esse assunto aparece tanto na formação?



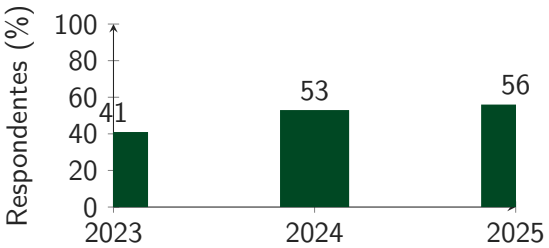
Na pesquisa Stack Overflow 2026, muitos participantes relataram trabalho com essas ferramentas no ano anterior.

Ideia principal

É uma evidência de presença no cotidiano dos respondentes. Não mede vagas de emprego nem a participação de cada ferramenta no mercado.

Fonte: Stack Overflow Developer Survey 2026. Questão: n=10.889; múltipla seleção; amostra voluntária. Docker/Compose: 6.440; Kubernetes: 2.836. Consulta: 15 out. 2026.

Contêineres passaram a sustentar mais aplicações



Parcela que usa contêineres na maioria ou em todas as aplicações de produção, no recorte da CNCF.

Em palavras simples

Para Ops, isso torna relevantes a manutenção de imagens, os limites de recursos e a observação de muitas execuções.

Fonte: CNCF Annual Survey 2025, fig. 6, p. 10. Bases: 522 (2023), 408 (2024), 365 (2025). Organizações usuárias; respostas "não sei" excluídas. Amostras diferentes; não é acompanhamento das mesmas empresas.

O que melhora para DevOps – e o que continua existindo

O contêiner ajuda a...	A equipe ainda precisa...
Repetir o ambiente da aplicação.	Controlar versões e configuração.
Iniciar outras cópias do mesmo pacote.	Verificar se há CPU, memória e dependências disponíveis.
Substituir uma execução com problema.	Entender a causa e proteger os dados.
Automatizar a publicação de versões.	Testar a mudança e acompanhar o resultado.

Dados importantes não devem depender apenas dos arquivos locais de uma execução que pode ser substituída. Bancos de dados e armazenamento persistente precisam de planejamento próprio.

Quando uma aplicação vira várias cópias

Uma réplica é outra execução da mesma aplicação. Na transportadora, é como abrir outro balcão capaz de calcular rotas, com o mesmo procedimento.

Escala vertical

Dar mais CPU ou memória à mesma execução. É ampliar a capacidade daquele balcão.

Escala horizontal

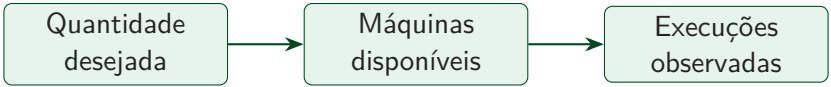
Executar mais réplicas e distribuir o trabalho entre elas. É abrir novos balcões.

Ideia principal

Réplicas funcionam melhor quando os pedidos podem ser atendidos independentemente. Se cada cópia guarda dados essenciais só para si, surgem problemas de consistência.

Quem cuida de todas essas execuções?

Com várias cópias em várias máquinas, alguém precisa escolher onde executá-las, perceber falhas, manter a quantidade desejada e permitir que os clientes as encontrem.



Em palavras simples

Orquestrar é coordenar esse conjunto continuamente. Na transportadora, é a organização que acompanha os balcões, a capacidade de cada unidade e a reposição do atendimento.

Ideia principal

A complexidade operacional cresce com a quantidade de partes. A próxima ferramenta responde a esse problema de coordenação.

Kubernetes: declarar o que deve continuar funcionando

Em palavras simples

Você descreve o que deseja manter em execução. O Kubernetes acompanha a situação e tenta aproximar o estado real desse objetivo.

Definição técnica

Kubernetes é uma plataforma de código aberto para gerenciar cargas de trabalho e serviços em contêineres, com configuração declarativa e automação.

Por exemplo: “mantenha três cópias desta aplicação”. Se uma delas desaparece, os controladores procuram restabelecer a quantidade desejada, desde que haja condições e recursos.

Fonte: Kubernetes, Overview. Consulta: 15 out. 2026.

Onde Kubernetes aparece na pesquisa CNCF?

82%

em produção

Esse é o percentual de uso de Kubernetes em produção entre usuários de contêineres no recorte do levantamento de 2025.

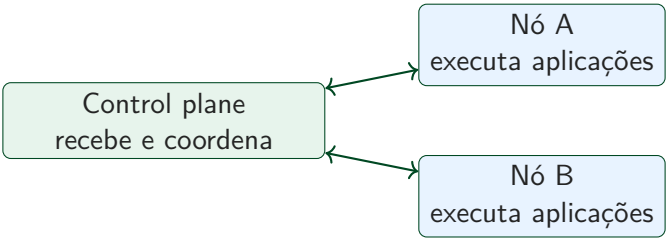
A pergunta teve 351 respostas de organizações usuárias, excluindo "não sei". A pesquisa completa teve 628 participantes familiarizados com cloud native.

Ideia principal

O resultado justifica conhecer a tecnologia. A decisão de adotá-la ainda depende do tamanho do problema e da capacidade de operar a plataforma.

Fonte: CNCF Annual Survey 2025, fig. 8, p. 11, e metodologia, pp. 21–22. Coleta: set. 2025; publicação: jan. 2026. Esse universo não representa todas as empresas.

Cluster: coordenação e máquinas de execução



Cluster é o conjunto. Nó é uma máquina física ou virtual que participa dele. O control plane guarda o estado desejado e coordena ações. Nos nós, agentes e runtimes mantêm as execuções.

Ideia principal

No laboratório, tudo cabe em uma máquina. Em produção, distribuir réplicas entre nós e zonas pode reduzir o impacto de uma falha comum.

Fonte: Kubernetes, Cluster Architecture. Consulta: 15 out. 2026.

Pod: a unidade que Kubernetes coloca para executar

Pod
um ambiente de rede e execução compartilhado

Em palavras simples

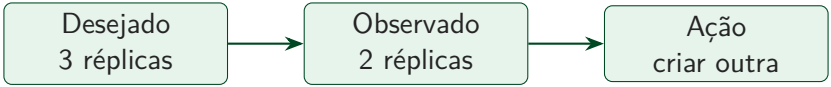
No nosso projeto, cada Pod contém uma cópia da aplicação de rotas. Ter três Pods significa ter três cópias disponíveis para atender pedidos.

Definição técnica

Pod é a menor unidade implantável do Kubernetes. Pode conter um ou mais contêineres que compartilham rede e podem compartilhar volumes. Um Pod é alocado em um nó.

Fonte: Kubernetes, Pods. Consulta: 15 out. 2026.

Deployment: manter a quantidade e a versão desejadas



Um Deployment descreve o modelo dos Pods e a quantidade pretendida. Seus controladores mantêm réplicas e coordenam mudanças de versão.

Em palavras simples

É a orientação registrada: "precisamos de três balcões com este procedimento". A coordenação compara essa orientação com o que está funcionando.

Ideia principal

Alterar um Pod isoladamente não é o mesmo que mudar o modelo da aplicação. Normalmente, administramos o Deployment que o controla.

Fonte: Kubernetes, Deployments. Consulta: 15 out. 2026.

Service: um endereço estável para encontrar as cópias



Pods podem ser substituídos e mudar de endereço. O Service oferece um ponto estável de acesso e encaminha conexões para os Pods selecionados e prontos.

Ideia principal

Não há garantia de divisão exatamente igual. Conexões persistentes podem continuar no mesmo Pod. No teste, novas conexões permitem observar a distribuição entre as réplicas.

Fonte: Kubernetes, Service; Virtual IPs and Service Proxies. Consulta: 15 out. 2026.

CPU e memória: aumentar réplicas também tem custo

Configuração do projeto	CPU total solicitada e limitada	Limite total de memória
1 Pod	500m = 0,5 CPU	128 MiB
2 Pods	1 CPU	256 MiB
3 Pods	1,5 CPU	384 MiB

Request é a necessidade declarada que orienta o agendamento. Limit é um teto. Limite de CPU pode causar contenção; exceder memória pode levar ao encerramento do contêiner.

Ideia principal

Mais Pods não criam capacidade física. Se o nó está cheio, novos Pods podem ficar pendentes. O gerador de carga e o próprio cluster também consomem recursos.

Fonte: Kubernetes, Resource Management. Consulta: 15 out. 2026.

Estar em execução não significa estar pronto

Readiness: pode receber tráfego?

Se a verificação falha, o Pod deixa de ser um destino pronto do Service. Isso evita enviar novos pedidos a uma instância indisponível.

Liveness: precisa reiniciar?

Se a verificação falha repetidamente, o kubelet reinicia o contêiner. O teste precisa refletir uma condição que o reinício possa resolver.

Uma verificação de inicialização, chamada startup probe, pode dar tempo para aplicações que demoram a iniciar. As três verificações têm papéis diferentes.

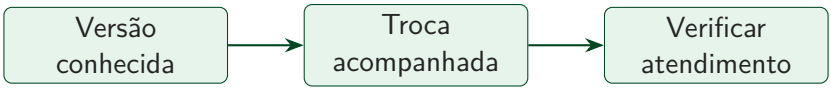
Ideia principal

Responder à verificação de saúde não garante que todas as funções do negócio estão corretas. Também precisamos observar o atendimento real.

Fonte: Kubernetes, Probes. Consulta: 15 out. 2026.

Publicar uma versão também é uma operação

Uma atualização gradual troca Pods antigos por novos conforme a estratégia configurada. A equipe acompanha se a nova versão está pronta e se os indicadores do serviço continuam aceitáveis.



Em palavras simples

Rollback é retornar a uma revisão anterior. Ele ajuda quando o problema está na mudança da aplicação, mas não desfaz automaticamente alterações em bancos de dados ou serviços externos.

Testes, compatibilidade e decisões de recuperação continuam sendo trabalho da equipe.

Fonte: Kubernetes, Deployments e rollback. Consulta: 15 out. 2026.

Kubernetes ajuda, mas não toma todas as decisões

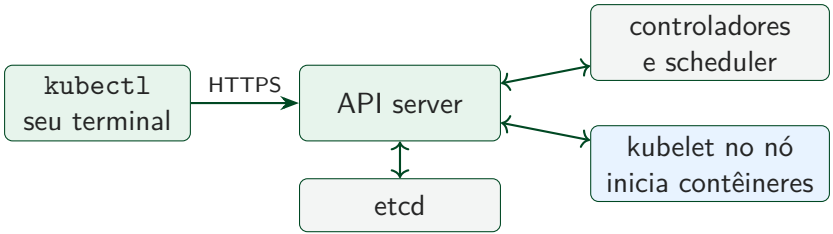
Pode automatizar	Depende do projeto e da operação
Reposição de execuções.	Corrigir um cálculo errado no código.
Distribuição e atualização de Pods.	Proteger dados e validar mudanças.
Ajuste de réplicas com autoscaling configurado.	Escolher métricas, metas, limites e capacidade.

O HPA pode ajustar a quantidade de Pods a partir de métricas. Ele precisa de configuração e de uma fonte de métricas. No experimento, a escala será manual para comparar condições conhecidas.

Ideia principal

Para um serviço simples, uma VM ou uma plataforma gerenciada pode ser suficiente. Conhecer Kubernetes inclui saber quando sua complexidade compensa.

kubectl: toda operação é uma chamada à API



O `kubectl` não liga contêineres. Ele envia pedidos à API; a API grava o desejado no etcd; controladores, scheduler e kubelet observam a API e agem. O mesmo comando funciona em um Minikube ou em um cluster de nuvem.

```
kubectl  <verbo>   <tipo>/<nome>        [opções]
kubectl  get       pods                  -o wide
kubectl  describe  deployment/rotas
kubectl  scale     deployment/rotas      --replicas=3
```

Fonte: Kubernetes, Command line tool (kubectl). Consulta: 15 out. 2026.

Contexto e namespace: onde o comando vai cair

```
$ kubectl config get-contexts
CURRENT   NAME          CLUSTER       NAMESPACE
          homologacao   homologacao   loja
*         ops-lab       ops-lab       ops-lab
          producao      producao      loja
```

Contexto

Cluster + usuário, guardados no kubeconfig. Define *qual* API recebe o comando.

Namespace

Divisão de nomes dentro do cluster. Organiza; sozinho, não isola.

Atenção

Um `kubectl delete` no contexto errado é um incidente real e frequente. Confira o contexto antes de alterar estado.

Manifesto: o estado desejado escrito em um arquivo

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: rotas
spec:
  replicas: 3
  selector:
    matchLabels: {app: rotas}
  template:
    metadata:
      labels: {app: rotas}
    spec:
      containers:
        - name: rotas
          image: rotas:1.0
```

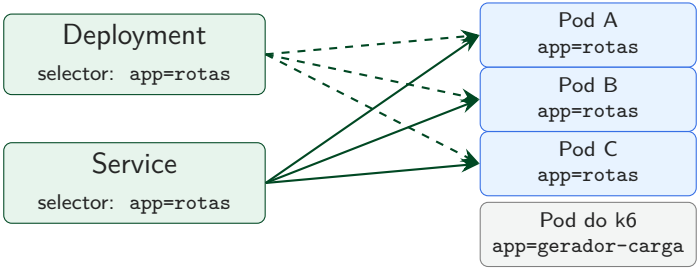
Campo	Papel
`apiVersion`	Versão da API que trata o objeto.
`kind`	Tipo: Deployment, Service, Job...
`metadata`	Nome, namespace e labels.
`spec`	O estado desejado.

`kubectl apply -f arquivo.yaml` envia o arquivo. O cluster preenche o `status` com o que observa.

Ideia principal

O arquivo vai para o Git, passa por revisão e é aplicado por pipeline. O histórico do repositório vira o histórico da infraestrutura.

Labels e selectors ligam as peças



Nenhum objeto aponta para outro pelo nome: Deployment e Service escolhem Pods pelas labels. O Pod do gerador tem outra label e fica de fora.

Atenção

Selector diferente das labels dos Pods: o Service é criado, mas fica sem destinos. Causa comum de "o serviço não responde".

Fonte: Kubernetes, Labels and Selectors. Consulta: 15 out. 2026.

Imperativo e declarativo

Imperativo: execute esta ação

```
kubectl create namespace ops-lab
kubectl scale deployment/rotas --replicas=3
kubectl set image deployment/rotas rotas=rotas
    :1.1
```

Rápido para investigar e experimentar. Não deixa registro do porquê.

Declarativo: o resultado deve ser este

```
# editar replicas: 3 no arquivo
kubectl apply -f k8s/deployment.yaml
```

Repetível: aplicar duas vezes dá o mesmo estado. O arquivo é revisável e versionado.

Ideia principal

Misturar os dois gera *drift*: um `scale` feito à mão é desfeito pelo próximo `apply` do arquivo. Ferramentas GitOps, como Argo CD e Flux, mantêm o cluster igual ao repositório.

Fonte: Kubernetes, Object Management. Consulta: 15 out. 2026.

Job e ConfigMap

	Deployment	**Job**
Objetivo	Manter um serviço rodando.	Executar uma tarefa até concluir.
Se o processo termina	Reinicia ou substitui.	Registra sucesso ou falha.
Exemplos	API, site, worker contínuo.	Migração, backup, teste de carga.

ConfigMap

Arquivos ou pares chave–valor entregues ao Pod como variáveis ou arquivos. Configuração fora da imagem.

Secret

Mesmo mecanismo para dados sensíveis. Por padrão é só base64; a proteção vem de RBAC e criptografia no etcd.

Fonte: Kubernetes, Jobs; ConfigMaps; Secrets. Consulta: 15 out. 2026.

Ler o estado de um Pod antes de agir

O que aparece	O que significa	Onde olhar
`Pending`	Ainda não foi colocado em um nó (recursos, regras).	`describe pod`
`ContainerCreating`	No nó, baixando imagem ou montando volumes.	`describe pod`
`Running, 0/1`	Processo rodando, readiness ainda falhando.	`describe pod`
`Running, 1/1`	Rodando e recebendo tráfego do Service.	–
`ImagePullBackOff`	Não conseguiu obter a imagem.	`describe pod`
`CrashLoopBackOff`	Processo termina e é reiniciado repetidamente.	`logs -previous`
`OOMKilled`	Passou do limite de memória.	`describe pod`
`Completed`	Terminou com sucesso (comum em Jobs).	`logs`

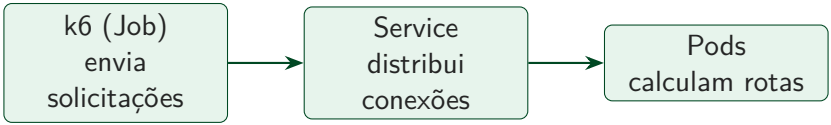
Ideia principal

`describe` e os eventos mostram o que o Kubernetes viu. `logs` mostram o que a aplicação escreveu. Os dois respondem perguntas diferentes.

Fonte: Kubernetes, Debug Pods. Consulta: 15 out. 2026.

Atividade: a transportadora precisa atender mais pedidos

O RotaExpress calcula a menor rota entre um depósito e nove pontos de entrega. Quando vários atendentes pedem rotas ao mesmo tempo, esse cálculo pode exceder a capacidade disponível.



Em palavras simples

A aplicação está pronta; a infraestrutura não. Vocês vão escrever os manifestos, publicar, verificar, provocar uma falha de implantação, testar carga e escalar, digitando cada comando. Os pontos são fictícios e as distâncias, em linha reta.

Ideia principal

Qual configuração atende melhor à mesma demanda? Qual é o custo adicional em recursos?

Atividade: oito grupos, cada um depende do anterior

	Grupo	Resultado	Comandos centrais
1	Cluster e contexto	Cluster ligado, namespace padrão	`minikube start`, `config`
2	Imagem	`rotas:1.0` testada e no nó	`docker build`, `image load`
3	Deployment	Pods mantidos pelo cluster	`apply`, `rollout status`
4	Service	Nome estável `rotas`	`get endpointslices`, `run`
5	Falha e rollback	Versão ruim revertida	`set image`, `rollout undo`
6	Teste de carga	ConfigMap, Job e taxa calibrada	`create configmap`, `logs`
7	Escala e medição	1, 2 e 3 Pods, três vezes cada	`scale`, `top`
8	Diagnóstico e decisão	Limite do nó, análise, limpeza	`describe`, `events`

Ideia principal

Testar a imagem antes do cluster e o Service antes da carga separa domínios de falha. Na medição, só a quantidade de Pods muda.

Atividade: o ciclo de cada medição

```
kubectl apply -f k8s/job-carga.yaml
kubectl wait --for=condition=Ready pod -l job-name=carga --timeout=180s
kubectl logs -f job/carga
kubectl logs job/carga > resultados/3pods-a.txt
kubectl delete job carga
```

Criar, esperar ficar pronto, acompanhar, guardar a evidência, limpar. Os logs pertencem ao Pod: apagar o Job antes de salvar perde a medição. Um Job concluído não roda de novo, por isso cada teste usa um Job novo.

Ideia principal

Status `Failed` com resumo no log significa que as metas do k6 não foram atingidas (código 99). É um resultado. Job sem resumo é falha de execução e não vira zero na tabela.

Atividade: o que entregar e explicar

Entrega	O que precisa mostrar
Manifestos	Pasta `k8s/` com Namespace, Deployment, Service e Job escritos pelo grupo.
Cenário	Taxa escolhida na calibração, recursos do cluster e por Pod.
Resultados	Tabela das nove execuções e medianas de 1, 2 e 3 Pods.
Evidências	Logs do k6, distribuição entre Pods, uso de CPU observado.
Conclusão	Configuração escolhida, custo em CPU reservada e explicação.

Metas do experimento: p95 abaixo de 800 ms, menos de 1% de falhas e nenhuma solicitação programada descartada pelo gerador. São critérios desta atividade.

Ideia principal

Se nenhuma configuração atingir as metas, explique por quê e proponha a próxima investigação.

Referências para continuar a leitura

Assunto	Fonte original
Pesquisas	Stack Overflow Developer Survey 2026: Technology Data e Methodology.
Adoção e desafios	CNCF Annual Cloud Native Survey 2025, publicado em 2026; figuras 6–8 e metodologia.
Operações	Google, Site Reliability Engineering: objetivos de serviço, trabalho repetitivo e monitoramento.
Entrega e DevOps	DORA: métricas de entrega de software e capacidades.
Contêineres	Docker Docs: contêineres e imagens.
Orquestração	Kubernetes: Concepts, especialmente Pods, Deployments, Services e recursos.
Operação	kubectl Quick Reference e depuração de Pods.
Teste de carga	Grafana k6: cenários, métricas e limites de aceitação.

Links clicáveis. Consulta: 15 out. 2026. As bases e limitações das pesquisas estão nos respectivos slides. Gráficos de pesquisas foram redesenhados a partir dos dados publicados.